# Tugas 2

## SISTEM PARALLEL DAN TERDISTRIBUSI

## Sinkronisasi dan Distributed Systems

**Dosen Pengampu :**
Riska Kurniyanto Abdullah, S.T., M.Kom.

**Disusun Oleh :**

Miftahul Fauzi Rifai 11231040

**Program Studi Informatika**
**Fakultas Sains dan Teknologi Informasi**
**Institut Teknologi Kalimantan**
**2026**

# Daftar Isi

**BAB 1**
**PENDAHULUAN** ..... 2
**1.1 Latar Belakang** ..... 2
**1.2 Tujuan** ..... 2
**BAB 2**
**ARSITEKTUR SISTEM** ..... 3
2.1 Komponen Sistem ..... 3
**2.2 Struktur Sistem** ..... 4
**BAB 3**
**IMPLEMENTASI SISTEM** ..... 5
**3.1 Distributed Lock Manager** ..... 5
**3.2 Distributed Queue System** ..... 5
**BAB 4**
**CRITICAL REVIEW** ..... 6
**4.1 Kelebihan Penelitian** ..... 6
**4.2 Keterbatasan atau Kekurangan** ..... 6
**4.3 Saran atau Masukan Pengembangan Penelitian** ..... 6
**BAB 5**
**KETERKAITAN DENGAN MATA KULIAH** ..... 8

# BAB 1
# PENDAHULUAN

## 1.1 Latar Belakang

Sistem terdistribusi merupakan pendekatan komputasi modern yang memungkinkan beberapa node bekerja secara bersamaan untuk menyelesaikan suatu permasalahan secara efisien dan terkoordinasi. Dalam implementasinya, sistem ini banyak digunakan pada berbagai bidang seperti cloud computing, microservices, dan pengolahan data skala besar. Namun, kompleksitas dalam sistem terdistribusi tidak hanya terletak pada pembagian tugas, melainkan juga pada bagaimana menjaga konsistensi data, sinkronisasi antar node, serta kemampuan sistem dalam menghadapi kegagalan (fault tolerance).

Untuk mengatasi tantangan tersebut, diperlukan mekanisme sinkronisasi yang andal dan efisien. Salah satu pendekatan yang umum digunakan adalah penerapan algoritma konsensus seperti Raft, yang memungkinkan setiap node dalam sistem mencapai kesepakatan terhadap suatu keadaan sistem secara konsisten. Selain itu, pengelolaan data dalam bentuk antrian terdistribusi (distributed queue) dan penyimpanan cache yang konsisten (cache coherence) juga menjadi komponen penting dalam menjaga performa dan keandalan sistem.

Berdasarkan kebutuhan tersebut, pada tugas ini dikembangkan sebuah sistem Distributed Synchronization System yang mencakup implementasi Distributed Lock Manager berbasis Raft, Distributed Queue menggunakan consistent hashing, serta Distributed Cache Coherence dengan protokol MESI. Sistem ini dirancang untuk berjalan pada beberapa node yang saling berkomunikasi dan menggunakan Redis sebagai media penyimpanan, sehingga diharapkan mampu mensimulasikan kondisi nyata dalam sistem terdistribusi serta memberikan pemahaman yang lebih mendalam terkait sinkronisasi dan koordinasi antar node.

## 1.2 Tujuan

Tujuan dari pengembangan sistem ini adalah:

- Mengimplementasikan koordinasi terdistribusi menggunakan algoritma Raft
- Menyediakan mekanisme lock terdistribusi
- Mengembangkan antrean terdistribusi dengan at-least-once delivery
- Mengimplementasikan protocol cache coherence
- Menyusun dokumentasi, pengujian, dan deployment sistem

# BAB 2
# ARSITEKTUR SISTEM

## 2.1 Komponen Sistem

Sistem Distributed Synchronization System dirancang menggunakan arsitektur berlapis (layered architecture) untuk memisahkan tanggung jawab setiap bagian sistem secara jelas. Komponen utama sistem terdiri dari:

a. Application Layer
   - Menyediakan API endpoints sebagai antarmuka antara client dan sistem
   - Menangani request seperti, Lock acquisition / release, Queue enqueue / dequeue, Cache read / write
   - Berfungsi sebagai entry point sistem

b. Distributed System Layer
   Lapisan inti yang mengelola logika sistem terdistribusi:
   - Lock Manager, mengelola distributed lock (shared & exclusive), menghindari konflik akses resource, mendukung deadlock detection
   - Queue Manager, menggunakan *consistent hashing* untuk distribusi pesan, mendukung multiple producer & consumer, menjamin *at-least-once delivery*
   - Cache Coherence (MESI), menjaga konsistensi cache antar node, mengelola state: Modified, Exclusive, Shared, Invalid, menangani invalidation & update propagation
   - Failure Detector, mendeteksi node failure, dan digunakan untuk menjaga stabilitas sistem

c. Consensus Layer (Raft)
   - Mengimplementasikan algoritma Raft untuk konsistensi data
   - Fitur utama, Leader election, Log replication, State machine synchronization, Term management
   - Menjamin bahwa semua node memiliki state yang sama

d. Communication Layer
   - Menggunakan aiohttp untuk komunikasi antar node
   - Menangani (Message passing, RPC antar node, Health check)

e. Persistence & Infrastructure
   - Redis (Digunakan sebagai persistence layer, Menyimpan data queue, lock, dan cache)
   - Docker (Containerization untuk deployment, Mendukung scalability. Mempermudah environment setup)

## 2.2 Struktur Sistem

Struktur proyek dibagi menjadi beberapa modul utama untuk menjaga modularitas dan kemudahan pengembangan:

a. src/nodes/

   Berisi implementasi utama node, Komponen:

   - lock_manager.py → logika distributed lock
   - queue_node.py → pengelolaan antrian
   - cache_node.py → cache coherence

b. src/consensus/

   - Implementasi algoritma Raft
   - Menangani (Leader election, Log replication, Konsistensi state antar node)

c. src/communication/

   - Mengatur komunikasi antar node
   - Komponen (Message passing, RPC handling, Failure detection)

d. src/utils/

   - Berisi utilitas pendukung sistem
   - Fungsi sebagai Konfigurasi (config.py) dan monitoring & metrics (metrics.py)

# BAB 3
# IMPLEMENTASI SISTEM

## 3.1 Distributed Lock Manager

Distributed Lock Manager dalam sistem ini diimplementasikan menggunakan algoritma konsensus Raft untuk memastikan konsistensi data antar node. Dengan pendekatan ini, hanya node yang berperan sebagai *leader* yang memiliki hak untuk memproses permintaan lock, sehingga dapat menghindari konflik dan inkonsistensi data. Sistem ini mendukung dua jenis mekanisme penguncian, yaitu *shared lock* yang memungkinkan beberapa node membaca resource secara bersamaan, serta *exclusive lock* yang membatasi akses hanya pada satu node untuk operasi tulis. Selain itu, sistem juga mengimplementasikan mekanisme *deadlock detection* untuk mendeteksi kondisi saling menunggu antar node dan mencegah terjadinya kebuntuan dalam proses distribusi. Seluruh implementasi komponen ini terdapat pada file src/nodes/lock_manager.py.

## 3.2 Distributed Queue System

Distributed Queue System dirancang untuk menangani distribusi pesan secara efisien dalam lingkungan multi-node dengan memanfaatkan algoritma *consistent hashing*. Pendekatan ini memungkinkan sistem untuk menentukan node tujuan penyimpanan pesan secara deterministik dan seimbang. Sistem ini mendukung banyak *producer* dan *consumer* secara bersamaan, sehingga mampu menangani beban kerja yang tinggi. Untuk menjaga keandalan data, setiap pesan yang masuk akan disimpan menggunakan mekanisme *message persistence*, sehingga tidak hilang meskipun terjadi kegagalan node. Selain itu, sistem menerapkan jaminan *at-least-once delivery*, yang memastikan setiap pesan akan diproses minimal satu kali. Mekanisme *recovery* juga disediakan untuk memulihkan data dan melanjutkan proses ketika terjadi kegagalan pada node. Implementasi komponen ini dapat ditemukan pada file src/nodes/queue_node.py.

## 3.3 Distributed Cache Coherence

Distributed Cache Coherence dalam sistem ini diimplementasikan menggunakan protokol MESI (Modified, Exclusive, Shared, Invalid) untuk menjaga konsistensi data cache antar node. Setiap node memiliki cache lokal yang dapat berinteraksi dengan node lain melalui mekanisme *invalidation* dan *update propagation* ketika terjadi perubahan data. Dengan pendekatan ini, sistem dapat memastikan bahwa data yang diakses oleh setiap node tetap konsisten meskipun berada dalam lingkungan terdistribusi. Selain itu, sistem juga menerapkan kebijakan penggantian cache menggunakan algoritma LRU (*Least Recently Used*) untuk mengoptimalkan penggunaan memori dan meningkatkan performa akses data. Implementasi dari komponen ini terdapat pada file src/nodes/cache_node.py.

## 3.4 Containerization

Untuk mendukung kemudahan deployment dan skalabilitas, sistem ini menggunakan teknologi containerization dengan Docker. Setiap node dalam sistem dikemas menggunakan Dockerfile tersendiri, sehingga memungkinkan lingkungan eksekusi yang konsisten di berbagai platform. Selain itu, digunakan *docker-compose* untuk mengatur orkestrasi antar container, termasuk konfigurasi jaringan dan dependensi antar layanan. Redis juga disertakan sebagai bagian dari konfigurasi container untuk mendukung kebutuhan persistence. Dengan pendekatan ini, sistem dapat dengan mudah dikembangkan dan diskalakan dengan menambahkan node baru sesuai kebutuhan.

# BAB 4
# API SISTEM

API yang tersedia dalam sistem:

- GET /health → pengecekan status node
- GET /status → status sistem
- POST /locks/acquire → mengambil lock
- POST /locks/release → melepas lock
- POST /queue/enqueue → menambah pesan
- POST /queue/dequeue → mengambil pesan
- GET/POST /cache/{key} → operasi cache

Dokumentasi lengkap tersedia pada:
docs/api_spec.yaml

# BAB 5
# HASIL PENGUJIAN BERDASARKAN DEMONSTRASI VIDEO

Berdasarkan hasil demonstrasi yang dilakukan, pengujian sistem mencakup beberapa aspek utama, yaitu pengujian komponen, pengujian integrasi, serta pengujian performa. Pada pengujian komponen, seluruh modul yang diuji menunjukkan hasil yang berhasil dijalankan dengan status aman (lulus), yang menandakan bahwa setiap bagian sistem telah berfungsi sesuai dengan yang diharapkan.

Selanjutnya, pada pengujian integrasi, sistem diuji dalam kondisi multi-node untuk memastikan keterhubungan antar komponen. Hasil pengujian menunjukkan bahwa proses integrasi berjalan dengan baik, meskipun terdapat dua peringatan (*warning*) kecil. Namun demikian, peringatan tersebut tidak mempengaruhi kinerja sistem secara keseluruhan, sehingga sistem tetap dinyatakan stabil dan aman untuk dijalankan.

Pada pengujian performa, sistem menunjukkan hasil yang memuaskan dengan kondisi operasional yang stabil. Hal ini diperkuat dengan pengujian menggunakan tools *Locust* yang mensimulasikan banyak pengguna secara bersamaan. Dalam skenario ini, sistem mampu menangani beban dengan sangat baik, ditunjukkan oleh waktu respon yang cepat serta antrian permintaan yang efisien.

Selain itu, tingkat kegagalan (*failure rate*) yang tercatat selama pengujian sangat rendah, bahkan hampir tidak signifikan. Hal ini menunjukkan bahwa sistem memiliki tingkat keandalan yang tinggi dalam menangani permintaan secara paralel dan mampu mempertahankan performa yang konsisten di bawah beban kerja yang meningkat.

# BAB 6
# TANTANGAN DAN PEMBELAJARAN

## 6.1 Sinkronisasi Heartbeat dan Election (Raft)

- Penentuan waktu *heartbeat* dan *election timeout* harus seimbang
- Jika terlalu cepat → memicu *unnecessary election*
- Jika terlalu lambat → memperlambat deteksi kegagalan leader
- Solusi: melakukan penyesuaian interval timeout agar sistem lebih stabil

## 6.2 Menghindari Split-Brain

- Terjadi ketika lebih dari satu node menganggap dirinya sebagai leader
- Dapat menyebabkan inkonsistensi data
- Ditangani dengan mekanisme quorum pada Raft (mayoritas suara)
- Node yang tidak memiliki mayoritas tidak dapat melakukan commit

## 6.3 Kesulitan Pribadi dalam Pengembangan

- Materi sistem terdistribusi tergolong kompleks dan mencakup banyak konsep sekaligus (consensus, networking, concurrency)
- Membutuhkan pemahaman mendalam terhadap algoritma seperti Raft yang tidak sederhana untuk diimplementasikan dari awal
- Proses debugging cukup sulit karena melibatkan banyak node yang berjalan secara bersamaan
- Sinkronisasi antar komponen sering menimbulkan error yang tidak mudah dilacak
- Waktu pengerjaan menjadi tantangan tersendiri karena kompleksitas tugas yang tinggi

## 6.4 Pembelajaran yang Diperoleh

- Memahami kompleksitas sistem terdistribusi secara praktis
- Mengetahui pentingnya konsensus dalam menjaga konsistensi data
- Belajar menangani kegagalan node dan kondisi jaringan
- Meningkatkan kemampuan debugging dan analisis sistem
- Mengembangkan kemampuan problem solving pada sistem berskala besar

# BAB 7
# PENUTUP

## 9.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem *Distributed Synchronization System* berhasil dikembangkan sesuai dengan tujuan yang telah ditetapkan. Sistem ini telah mengimplementasikan beberapa komponen utama, yaitu *Distributed Lock Manager* berbasis algoritma Raft, *Distributed Queue System* dengan pendekatan *consistent hashing*, serta *Distributed Cache Coherence* menggunakan protokol MESI. Selain itu, sistem mampu berjalan dengan baik pada konfigurasi multi-node dan menunjukkan performa yang stabil dalam berbagai skenario pengujian. Mekanisme konsensus yang diterapkan juga terbukti mampu menjaga konsistensi data antar node, sehingga sistem dapat dikategorikan sebagai sistem yang cukup andal, konsisten, dan memiliki tingkat skalabilitas yang baik dalam lingkungan terdistribusi.

## 9.2 Lampiran Dokumen/Tautan Terkait

| Jenis Dokumen | Tautan (Link) |
|---|---|
| **Repository GitHub** | https://github.com/Miftahul-Fauzi-Rifai/Tugas-2-Sistem-Pararel-dan-Terdistribusi |
| **Video Demonstrasi (Youtube)** | https://youtu.be/l8kzPzEJFP0 |